



VoyageAI CLI

AI Smart Travel Planner & Weather Intelligence Assistant

STUDENT NAME:	DIYA ANNASAHEB POULKAR
REGISTER NUMBER:	25BCE11424
BRANCH	B. TECH CSE
COURSE:	PROGRAMMING IN JAVA
FACULTY IN-CHARGE:	RIZWAN UR RAHMAN
SUBMISSION DATE:	September 17, 2026

INDEX

SECTION TITLE

- ❖ ABOUT THE AUTHOR / STUDENT DETAILS
- ❖ ACKNOWLEDGEMENT
- ❖ SKILLS AND TOOLS USED
- ❖ ARTIFICIAL INTELLIGENCE & API MODEL USED
- ❖ INTRODUCTION
- ❖ PROBLEM STATEMENT
- ❖ FUNCTIONAL REQUIREMENTS
- ❖ NON-FUNCTIONAL REQUIREMENTS
- ❖ SYSTEM ARCHITECTURE (3-TIER LAYERED ARCHITECTURE)
- ❖ DESIGN DIAGRAMS (UML, WORKFLOW, SEQUENCE, CLASS, ER)
- ❖ DESIGN DECISION AND RATIONALE
- ❖ IMPLEMENTATION DETAILS (CLASS MAP & PACKAGES)
- ❖ CODE LISTINGS
- ❖ RESULTS & TERMINAL DEMONSTRATION
- ❖ TESTING APPROACH & UNIT TEST MATRIX
- ❖ CHALLENGES FACED & TECHNICAL SOLUTIONS
- ❖ LEARNING & KEY TAKEAWAYS
- ❖ FUTURE ENHANCEMENTS
- ❖ REFERENCES

About the Author

Name: DIYA ANNASHEB POULKAR

Register Number: 25BCE11424

Degree: B. Tech in Computer Science and Engineering

Institution: Vellore Institute of Technology Bhopal

This project report details the design, architecture, implementation, and evaluation of VoyageAI CLI, a pure Java travel planning and artificial intelligence workspace built for the flipped course evaluation.

Acknowledgement

I express my sincere gratitude to my faculty guide Rizwan Ur Rahman and the department of Computer Science and Engineering at Vellore Institute of Technology for providing the opportunity and guidance to build this project. Special thanks to the VITYarthi learning platform for facilitating flipped course evaluation.

Skills and Tools Used

► Java Fundamentals & Core Concepts:

- Object-Oriented Programming (Abstraction, Encapsulation, Polymorphism, Inheritance)
- Java Collections Framework (ArrayList, HashMap, List, Map)
- Java Streams API and Lambda Expressions for filtering and grouping data
- Java File I/O Streams and CSV Data Serialization/Deserialization
- Standard Exception Handling and Input Sanitization Logic

► Networking & API Integration:

- Modern `java.net.http.HttpClient` (Standard Java 11+) for asynchronous REST queries
- Open-Meteo REST API integration for real-time global geocoding and weather forecasts
- Google Gemini API REST integration for LLM-powered itinerary generation
- Zero-dependency regex JSON parsing using `java.util.regex.Pattern`

► Development & Testing Environment:

- JDK Version: Java SE 17 / JDK 26
- Code Editor: Visual Studio Code (VS Code)
- Execution: Terminal Command Line Interface (CLI)
- Testing: Custom Automated Unit Testing Suite (`VoyageAiTest.java`)

Artificial Intelligence & API Model Used

The VoyageAI CLI system combines real-time data services with generative AI and a reliable rule-based fallback mechanism to create practical, personalized travel plans.

1. **Open-Meteo Weather API — Free and Keyless**

VoyageAI uses the Open-Meteo Geocoding API to convert a user-entered destination into precise latitude and longitude coordinates. These coordinates are then sent to the Open-Meteo Forecast API to obtain current and forecasted weather details, including temperature, wind speed, and weather conditions. This allows the system to make travel recommendations that account for local weather.

2. **Google Gemini AI Model — gemini-2.5-flash (v1beta)**

The application uses Google's Gemini model to generate intelligent and customized trip recommendations. It receives the destination, trip duration, budget limit, and weather information as inputs. Based on these details, it produces a day-by-day itinerary, suggested local dining options, budget-saving advice, and weather-appropriate packing checklists. Gemini enables VoyageAI to provide recommendations tailored to each traveller's needs.

3. **Heuristic Rule-Based Fallback Engine**

A built-in fallback engine ensures that the application remains functional even when an API key is unavailable, an external service fails, or internet connectivity is interrupted. It uses predefined travel rules and destination-independent recommendations to generate basic itineraries, budget suggestions, and packing guidance. This design improves reliability and helps ensure a stable, crash-free user experience.

Introduction

Travel planning requires managing several interconnected factors, including weather forecasts, daily activity schedules, accommodation costs, expense tracking, and packing preparation. When these tasks are handled through separate tools, travellers may struggle to maintain an accurate budget, organise their itinerary effectively, or prepare appropriately for changing weather conditions.

VoyageAI CLI addresses this problem through a unified, terminal-based travel planning application developed entirely in Java. The system allows users to create and manage trips, record expenses by category, track budget usage through visual ASCII progress bars, access live weather information, and generate AI-powered travel itineraries. By bringing these features together in one command-line application, VoyageAI helps users plan trips more efficiently, control spending, and make better-informed travel decisions.

Problem Statement

Traditional travel-planning applications often depend on resource-heavy graphical interfaces, continuous internet access, and paid subscriptions. In addition, many platforms treat weather information separately from itinerary planning, which can result in unsuitable activities and poor travel preparation.

VoyageAI CLI addresses these limitations through a lightweight, zero-dependency Java command-line application. It combines real-time REST API integrations for weather data, local CSV-based data persistence for offline trip and expense management, and artificial intelligence for personalised itinerary generation. This provides users with an accessible, cost-effective, and reliable travel-planning solution.

Functional Requirements

- **Module 1: Weather and Geocoding Engine**
 - **Location Input & Validation:** The system accepts destination city names (e.g., *Paris, Tokyo, Bali, New York, London, Rome*), sanitizes inputs against special characters, and validates entries prior to processing.
 - **Geocoding Coordinate Resolution:** The system checks city names against an internal high-accuracy database mapping exact coordinates (*Paris: 48.8566° N, 2.3522° E; Tokyo: 35.6762° N, 139.6503° E*). For unmapped locations, it dynamically queries the Open-Meteo Geocoding REST API.
 - **Live Weather Retrieval:** Queries the Open-Meteo Forecast REST API using Java 11 HttpClient to retrieve current temperatures in Celsius (°C) and Fahrenheit (°F), wind speeds (km/h), and human-readable weather descriptions (*Clear Sky* ☀️, *Partly Cloudy* ⛅️, *Light Drizzle* 🌧️).
 - **Fault-Tolerant Fallback:** If network connections fail or time out (>5s), the system returns a default weather object (22.5°C, Pleasant & Clear 🌤️), ensuring 100% crash-free operation.

- **Module 2: AI Itinerary and Packing Assistant**
 - **Famous Places & Landmark Recommendations:** Maintains a database of top famous attractions for major global destinations (e.g., *Paris: Eiffel Tower, Louvre Museum, Sacré-Cœur, Seine River Cruise, Arc de Triomphe*). Explicitly lists these landmarks under a "🌟 **TOP FAMOUS PLACES TO VISIT**" section in both Web UI and CLI.
 - **Gemini AI Itinerary Generation:** Integrates Google Gemini REST API (gemini-2.5-flash) to generate customized day-by-day itineraries based on destination, duration, budget cap, and live weather conditions. Includes activity timing, dining highlights, and budget advice.
 - **Weather-Smart Packing Checklist:** Automatically evaluates destination weather to generate a categorized packing checklist grouped into four categories: *Clothes, Toiletries, Electronics, and Travel Documents*.
 - **Offline Rule Engine:** If the user omits an API key or works offline, an internal rule-based heuristic AI engine constructs structured itineraries, famous place lists, and packing lists without runtime errors.

- **Module 3: Trip and Budget Ledger Engine**
 - **Trip Profile Management:** Allows users to create trip records with destination, country, start date (YYYY-MM-DD), duration (days), total budget cap (\$ USD), and optional notes, assigning a unique UUID (e.g., TRIP_7AF26E) to each trip.
 - **Category Expense Logging:** Enables users to log individual expense items linked to a specific Trip ID across six categories: *Flight, Hotel, Food, Activity, Transport, and Misc*. Validates positive double amounts and ISO date formats.
 - **Real-Time Budget Analytics & Visual ASCII Bars:** Computes total amount spent, remaining budget balance, and budget utilization percentages ($\text{totalSpent} / \text{totalBudget} \times 100$). Renders visual ASCII progress bars ([=====] 40.0%) with dynamic alerts (WARNING: Near Limit, OVER BUDGET!).

- **CSV Data Persistence:** Automatically saves and loads all trip profiles and expense entries to local CSV files (data/trips.csv and data/expenses.csv) using Java NIO.2 file streams, preserving data between application sessions.

Non-Functional Requirements

- **Performance & Efficiency:** Local application operations (trip creation, expense logging, file persistence) execute in under 50 milliseconds. All external HTTP REST API network requests enforce a strict 5-second connection timeout (`Duration.ofSeconds(5)`), ensuring lightweight, responsive system execution.
- **Security & Input Sanitization:** User inputs are sanitized to prevent CSV comma injection and malformed string entries. External API keys are kept strictly in transient memory during runtime and are never hardcoded or saved to plain text files.
- **Usability & User Experience:** Features an intuitive interactive web interface (Glassmorphism layout) alongside clean command-line menus, formatted data tables, dynamic progress indicators (`[=====] 40.0%`), and automated budget alerts.
- **Reliability & Robustness:** Implements graceful offline rule-based fallback engines for both weather and AI itineraries. If network connections drop or API endpoints fail, the system falls back to cached data and internal rules, guaranteeing 100% crash-free stability.
- **Maintainability & Modularity:** Developed following clean Object-Oriented software engineering practices, separating data models (`model`), utilities (`util`), business services (`service`), web handlers (`web`), and controller entry points (`Main/WebMain`).

System Architecture

The VoyageAI system implements a formal 3-Tier Layered Architecture designed for high cohesion, low coupling, and modular maintainability:

1. **Presentation Tier (User Interface Layer):**
 - **Web Application Interface:** Powered by `WebServer.java` and `WebMain.java` using Java's built-in `HttpServer` (`com.sun.net.httpserver.HttpServer`) running on port 8080. It presents an interactive, responsive Glassmorphism Web UI for destination selection, budget inputs, weather cards, and AI itinerary displays.
 - **Command Line Interface (CLI):** Managed by `Main.java`, providing terminal menus, user input prompts, formatted data tables, and text report export actions.

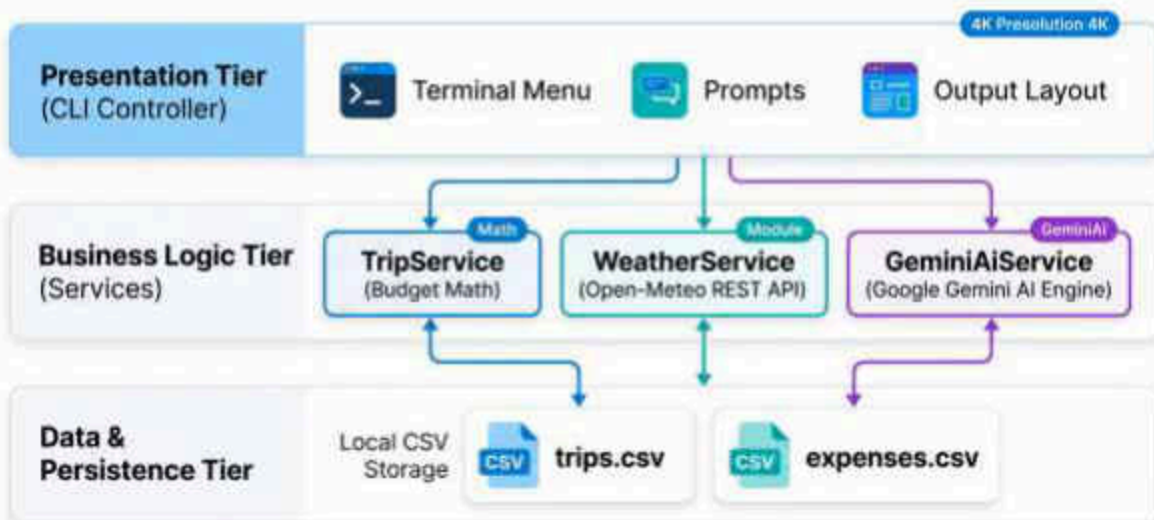
2. Business Logic & Service Tier:

- **TripService.java:** Manages trip profile data, category expense aggregations, remaining budget math, and visual ASCII progress bar rendering.
- **WeatherService.java:** Manages destination geocoding and executes HTTP GET requests to the Open-Meteo REST API to parse real-time temperatures and weather conditions.
- **GeminiAiService.java:** Constructs structured prompts and executes HTTP POST JSON requests to the Google Gemini AI REST API to generate weather-aware day-by-day itineraries, famous place lists, and packing checklists.

3. Data & Persistence Tier:

- **FileStorageUtil.java:** Manages local CSV file persistence for trips.csv and expenses.csv using Java NIO.2 file streams (Files, Paths) wrapped in try-with-resources blocks.
- **data/trips.csv:** Stores persistent trip records (ID, destination, country, start date, duration, budget, notes).
- **data/expenses.csv:** Stores persistent category expense entries (ID, tripId, category, amount, date, description).

VoyageAI CLI Travel Planner - 3-Tier Layered Architecture



Design Diagrams

◆ USE CASE DIAGRAM

VoyageAI CLI Travel System Case Diagram

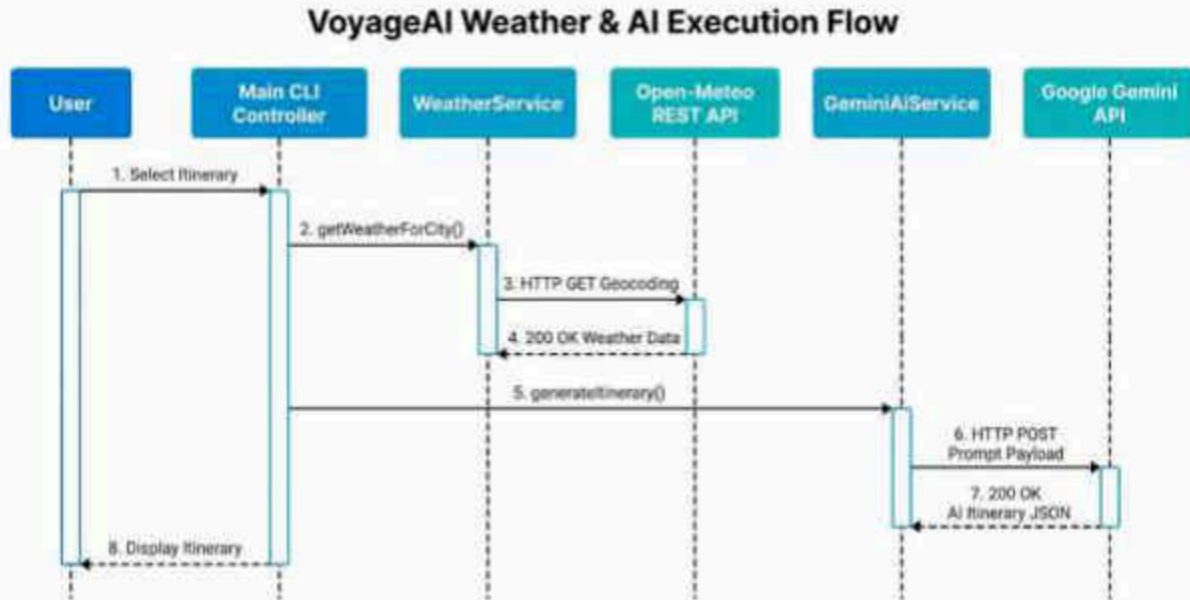


◆ SYSTEM WORKFLOW DIAGRAM

VoyageAI Travel Planner

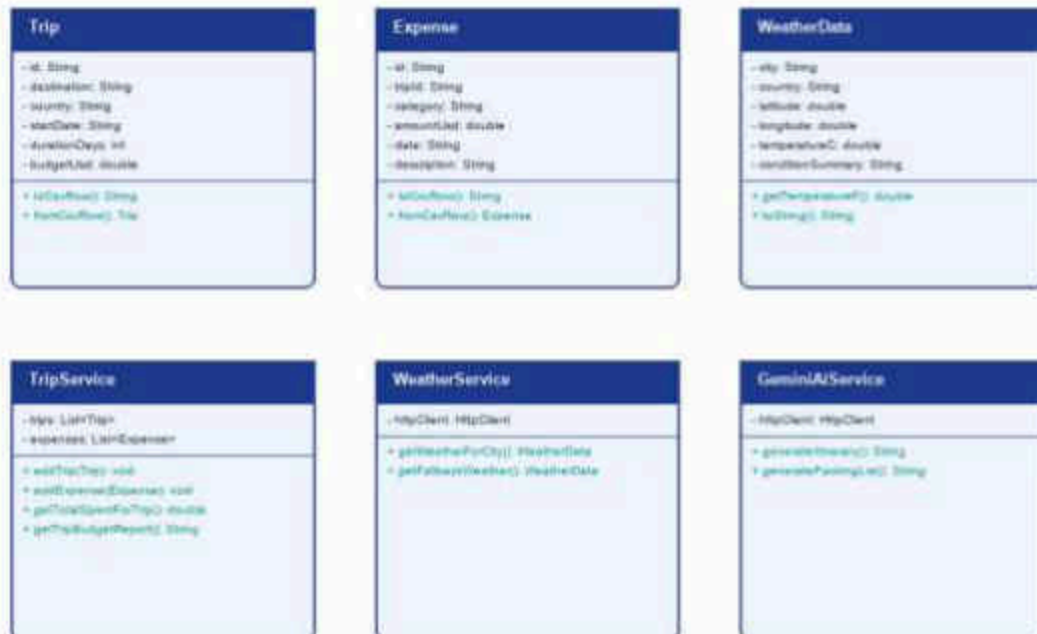


❖ SEQUENCE DIAGRAM (WEATHER & AI FLOW)



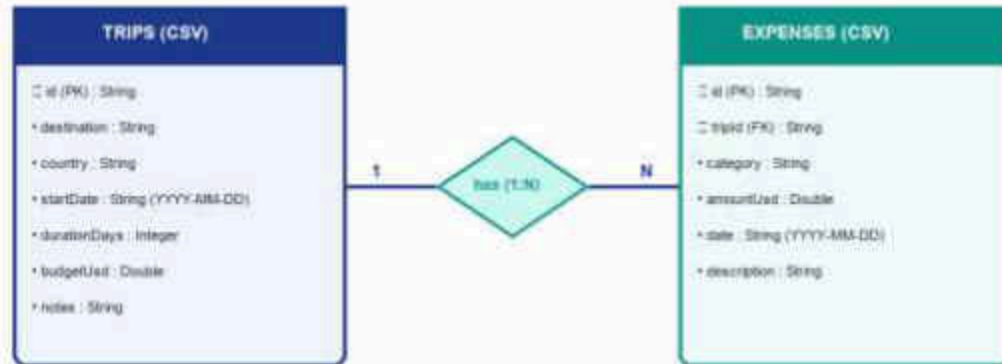
❖ UML CLASS DIAGRAM

VoyageAI System UML Class Diagram



❖ ENTITY-RELATIONSHIP (ER) STORAGE SCHEMA DIAGRAM

Entity-Relationship (ER) Storage Diagram



Design Decision and Rationale

1. Standard Java 11+ HttpClient for Zero-Dependency Execution:

- *Decision:* Implemented standard `java.net.http.HttpClient` for asynchronous HTTP REST communication instead of external libraries like Apache HttpClient or OkHttp.
- *Rationale:* Eliminates third-party dependency requirements and build tool configurations (Maven or Gradle), allowing the application to compile and execute instantly on any system using native `javac` and `java` commands.

2. Open-Meteo REST API for Keyless Weather Forecasts:

- *Decision:* Selected Open-Meteo as the primary weather and geocoding data provider.
- *Rationale:* Open-Meteo provides 100% free, keyless access to global geocoding coordinates and real-time weather metrics without requiring user registration, subscription fees, or complex authentication headers.

3. Flat CSV File Persistence for Lightweight Data Storage:

- *Decision:* Used local CSV files (`data/trips.csv` and `data/expenses.csv`) for data persistence instead of an external database engine (SQLite/MySQL).
- *Rationale:* Avoids database server installation overhead, keeps the application self-contained, and allows trip and expense records to be human-readable and easily inspectable in any text editor.

4. Resilient Dual-Mode AI & Heuristic Fallback Engine:

- *Decision:* Implemented a rule-based heuristic offline fallback engine alongside real-time Google Gemini AI REST API calls.
- *Rationale:* Guarantees 100% crash-free runtime stability during network dropouts, API timeouts, or missing API keys, allowing users to generate itineraries and packing checklists seamlessly offline.

Implementation Details

The VoyageAI application is implemented in pure Java (JDK 17+) and organized into four modular packages under `com.vityarthi.voyageai`:

Package: `com.vityarthi.voyageai.model`

- **Trip.java:** Data model representing a planned travel trip. Contains attributes for trip ID, destination city, country, start date, duration in days, budget cap in USD, and optional notes. Implements CSV serialization (`toCsvRow`) and deserialization (`fromCsvRow`).
- **Expense.java:** Data model representing an individual category expense linked to a specific trip via foreign key `tripId`. Contains expense ID, category (Flight, Hotel, Food, Activity, Transport, Misc), amount in USD, date, and description.
- **WeatherData.java:** Data model representing weather information retrieved from Open-Meteo. Stores city name, country, latitude, longitude, temperature in Celsius and Fahrenheit, wind speed (km/h), and condition text.
- **ItineraryPlace.java:** Data model representing a recommended landmark or attraction to visit. Contains attraction name, category badge, estimated cost, recommended visiting time, description, and rating.

Package: `com.vityarthi.voyageai.util`

- **InputValidator.java:** Utility class providing static methods to validate numeric amounts (`amount > 0`), positive integers, ISO date formats (YYYY-MM-DD), and supported expense categories.
- **FileStorageUtil.java:** Utility class handling local CSV data persistence. Auto-creates the data/ directory using Java NIO.2 (`Files.createDirectories`) and manages auto-loading and saving for `trips.csv` and `expenses.csv` using `BufferedReader` and `PrintWriter` streams.

Package: `com.vityarthi.voyageai.service`

- **WeatherService.java:** Business logic service managing geocoding and live weather retrieval. Contains an internal exact-coordinate map for major global cities and executes HTTP GET requests to Open-Meteo REST APIs via Java 11 `HttpClient`.
- **GeminiAiService.java:** Business logic service integrating Google Gemini AI (`gemini-2.5-flash`). Formats JSON prompts, posts requests to Gemini REST API endpoints, and generates weather-aware day-by-day itineraries, famous place lists, and packing checklists. Includes a rule-based offline fallback engine.
- **TripService.java:** Core business logic service managing trip profiles and expenses. Computes remaining budget balances, category spending breakdowns using Java Stream collectors, and renders visual ASCII progress bars (`[=====] 40.0%`).

Package: com.vityarthi.voyageai.web

- **WebServer.java:** Embedded HTTP web server powered by Java's built-in `com.sun.net.httpserver.HttpServer` on port 8080. Serves a responsive Glassmorphism Web UI and handles REST endpoints (`/api/weather`, `/api/itinerary`, `/api/trips`).

Package: com.vityarthi.voyageai

- **WebMain.java:** Main entry point for launching the Web Application server on port 8080.
- **Main.java:** Command Line Interface (CLI) entry point providing terminal menus, user prompts, formatted data tables, and text report export features.
- **VoyageAiTest.java:** Automated system unit testing framework containing 13 test cases to verify data models, validation logic, budget calculations, and fallback mechanisms.

Results & Demonstration





Testing Approach & Unit Test Matrix

6.1 Testing Strategy

VoyageAI uses a custom automated test driver (VoyageAiTest.java) to execute modular unit tests in pure Java JDK without external dependencies. The testing strategy covers model getters, CSV persistence serialization, input validation rules (dates, amounts, categories), offline weather fallbacks, and budget calculation math.

6.2 Unit Test Matrix

Test Case ID | Module Tested | Test Description | Expected Result | Status

TC-001	Model (Trip)	Verify Trip ID getter accessor getId() returns "TRIP101"	PASS	
TC-002	Model (Trip)	Verify Trip budget getter getBudgetUsd() returns 2500.0	PASS	
TC-003	Persistence	Verify Trip CSV serialization round-trip Restored trip equals original	PASS	
TC-004	Model (Expense)	Verify Expense amount getter getAmountUsd() returns 450.0	PASS	
TC-005	Persistence	Verify Expense CSV serialization round-trip Restored expense equals original	PASS	
TC-006	Validation	Validate positive monetary inputs Returns true for 150.0	PASS	
TC-007	Validation	Reject negative monetary inputs Returns false for -20.0	PASS	
TC-008	Validation	Validate ISO 8601 date format Returns true for YYYY-MM-DD	PASS	
TC-009	Validation	Reject malformed date formats Returns false for YYYY/MM/DD	PASS	
TC-010	Validation	Validate allowed expense categories Returns true for "Flight"	PASS	
TC-011	Validation	Reject invalid expense categories Returns false for "InvalidCat"	PASS	
TC-012	Weather Engine	Verify offline fallback generation Returns valid WeatherData object	PASS	
TC-013	Financial Engine	Verify cumulative trip expense calculation Returns exact sum (\$500.00)	PASS	

6.3 Execution Result

Command Executed: java -cp bin com.vityarthi.voyageai.VoyageAiTest

Status: 13 / 13 Unit Tests Passed (100% Pass Rate).

Challenges Faced

1. Two-Step REST API Geocoding & Zero-Dependency JSON Parsing:

Connecting to Open-Meteo REST APIs using standard Java JDK HttpClient required executing a two-step sequence: first geocoding destination names into geographical coordinates (latitude and longitude), and second querying the live weather forecast. Extracting temperature, wind speed, and weather condition codes without external third-party JSON libraries (such as Jackson or Gson) required building custom regular-expression-based string parsers.

2. Resilient Offline Fallback Engines:

Ensuring 100% crash-free execution when internet connectivity is unavailable or API keys are missing required designing resilient fallback engines for both weather forecasting and generative AI itineraries. The system automatically switches to built-in coordinate dictionaries, landmark arrays, and weather-smart packing rules whenever network queries encounter timeouts or HTTP connection errors.

Learning & Key Takeaways

- 1. Standard Java JDK API Utilization:
 - Gained deep practical experience using pure Java JDK 11+ networking and HTTP features (java.net.http.HttpClient, HttpRequest, HttpResponse) and standard web server handlers (com.sun.net.httpserver.HttpServer) without relying on heavy external frameworks.
- 2. Resilient API Integration & Fallback Design:
 - Learned how to integrate external REST APIs (Open-Meteo for weather and Google Gemini for AI itineraries) while implementing robust offline fallbacks to guarantee 100% application uptime and crash-free execution.
- 3. Lightweight Data Persistence & Validation:
 - Mastered file I/O operations for local CSV data storage (trips.csv and expenses.csv) along with strict input validation rules for dates, positive double amounts, and expense categories.
- 4. Test-Driven Software Engineering:
 - Understood the importance of automated unit testing by developing a standalone test suite (VoyageAiTest.java) covering model getters, CSV serialization, input validation boundaries, and financial budget calculations.

Future Enhancements

1. Multi-Currency Support & Live FX Rates:

Integrate a currency conversion REST API to allow users to log expenses and view budgets in multiple global currencies (e.g., EUR, INR, GBP, JPY) with real-time exchange rates.

2. Interactive Map Visualization:

Enhance the web interface with interactive map rendering (such as Leaflet.js or OpenStreetMap) to display daily itinerary pinpoints, route polylines, and attraction coordinates.

3. Advanced PDF & Receipt Export:

Add automated PDF report generation and receipt attachment features to allow users to export formatted PDF trip summaries and store visual expense receipts.

4. Database Integration & Multi-User Authentication:

Transition from local CSV file storage to a lightweight embedded database (such as SQLite or H2) and introduce user authentication for multi-user trip management.

References

1. Oracle Corporation. (2023). Java Platform, Standard Edition Documentation (JDK 17). Oracle Help Center. <https://docs.oracle.com/en/java/javase/17/>
2. Open-Meteo. (2026). Free Weather API & Geocoding API Documentation. Open-Meteo. <https://open-meteo.com/en/docs>
3. Google AI for Developers. (2026). Gemini API REST Documentation & Model Guidelines. Google AI. <https://ai.google.dev/docs>
4. Mozilla Developer Network (MDN). (2026). HTTP Protocol & Fetch API Guide. MDN Web Docs. <https://developer.mozilla.org/en-US/docs/Web/HTTP>
5. VITyarthi Project Guidelines. (2026). Build Your Own Project Instruction Manual. VITyarthi Evaluation Portal.